

Trabajo Práctico: Optimización Aplicada a la Distribución Logística

Diseño de un Módulo de Shelf Selection para un Sistema Robótico G2P

12 de octubre de 2025

Índice

1. Introducción y Contexto de Negocio	2
2. Descripción del Problema	2
2.1. Objetivo Principal	2
2.2. Componentes y Supuestos del Sistema	2
2.3. Función Objetivo y Criterios de Optimización	3
3. Consigna del Trabajo Práctico	4
3.1. Fase 1: Análisis del Problema y Requerimientos	4
3.2. Fase 2: Exploración de Datos y Enfoques de Solución	4
3.3. Fase 3: Propuesta de Arquitectura	5
3.4. Fase 4: Propuesta de un Algoritmo para el SS	5
3.5. Fase 5: Métricas de Evaluación de Performance	5
4. Entregables	5

1. Introducción y Contexto de Negocio

En el vertiginoso mundo del e-commerce, la eficiencia de los centros de distribución (Fulfillment Centers) es un pilar fundamental para el éxito del negocio. Una de las tecnologías más disruptivas en este ámbito son los sistemas robóticos **Goods-to-Person (G2P)**, donde robots autónomos transportan estanterías (racks) desde un área de almacenamiento hasta estaciones de trabajo donde los operarios realizan el picking de los productos.

La arquitectura de software que gobierna estos sistemas se estructura en tres capas principales:

- **WMS (Warehouse Management System):** La capa superior, responsable de la gestión del inventario y la orquestación general de las tareas del almacén.
- **WES (Warehouse Execution System):** La capa intermedia, que traduce las directivas del WMS en tareas ejecutables por la capa inferior, optimizando los flujos de trabajo en tiempo real.
- **RCS (Robot Control System):** La capa de más bajo nivel, que gestiona la navegación y ejecución de movimientos de la flota de robots.

Este trabajo práctico se enfoca en un componente crítico del **WES** dentro del proceso de **Outbound** (preparación de pedidos vendidos): el módulo de **Shelf Selection (SS)**. La función principal del SS es decidir de manera inteligente qué estanterías deben ser transportadas a las estaciones de picking para procesar las órdenes pendientes, con el objetivo de maximizar la eficiencia global del sistema.

El desafío consiste en desarrollar una versión simplificada pero robusta de este módulo, aplicando conceptos de optimización combinatoria.

2. Descripción del Problema

Se debe diseñar e implementar un algoritmo para el módulo de **Shelf Selection (SS)** que opere bajo un conjunto de supuestos y requerimientos simplificados.

2.1. Objetivo Principal

El objetivo es construir un módulo SS que, al ser invocado, seleccione un conjunto de órdenes del backlog y las asigne a racks específicos para ser procesados. La meta es maximizar la densidad de picking, minimizando al mismo tiempo el esfuerzo de movimentación de los robots.

2.2. Componentes y Supuestos del Sistema

1. **Foto de Stock:** Se cuenta con un archivo `stock.json` que representa una foto del inventario en un momento inicial. Este archivo detalla la ubicación de cada ítem (SKU) en el almacén, especificando el rack, la **cara del rack (numerada del 1 al 4)** y la cantidad de unidades disponibles en dicha ubicación.
2. **Backlog de Órdenes:** Se dispone de un backlog de órdenes **mono-ítem** (cada orden contiene un solo tipo de producto y una sola unidad). Cada orden en el backlog tiene la siguiente información:

- **order_id:** Identificador único de la orden.
- **item_id:** Identificador del producto solicitado.
- **creation_date:** Fecha y hora de creación de la orden.
- **due_date:** Fecha y hora límite para procesar la orden (SLA - Service Level Agreement).

3. **Interacción con el Task Manager:** El módulo SS no opera de forma continua, sino que es invocado periódicamente como parte de un proceso más complejo que luego ejecuta a un **Task Manager (TM)**. El TM será el componente responsable de convertir las órdenes seleccionadas por el SS en un conjunto de tareas ordenadas, con origen y destino (posición original del rack y estaciones de trabajo), que será entregado al RCS para su ejecución.

- **Ciclo de Invocación:** El SS es llamado cada 5 minutos.
- **Input para el SS:** En cada llamada, al SS se tiene acceso a:
 - a) El listado completo de las órdenes en el backlog que todavía no fueron procesadas
 - b) Un listado de tareas **pendientes** del ciclo anterior que no fueron procesadas por el TM y deben ser priorizadas.
 - c) La cantidad **N** de órdenes que se necesitan procesar en el TM en el siguiente ciclo (**N** es input del SS y determina el tamaño del output en una determinada ejecución).
- **Output del SS:** El SS debe devolver un "pool de tareas" que agrupe tanto las órdenes **pendientes** del ciclo anterior como nuevas órdenes seleccionadas del backlog. El total de órdenes devueltas debe ser **N** (o todas las disponibles si el backlog es menor). Las órdenes deben estar agrupadas por **cara de rack**. Por ejemplo: { (Rack_1, Cara_2): [orden_1, orden_2], (Rack_5, Cara_4): [orden_3, orden_4, orden_5], ... }. El output deberá contener de manera mandatoria las tareas pendientes no procesadas en el ciclo anterior por el TM.

2.3. Función Objetivo y Criterios de Optimización

El diseño del algoritmo SS debe optimizar los siguientes criterios, en orden de prioridad:

1. **Maximizar el "Picks Per Face":** El objetivo primario es agrupar la mayor cantidad posible de órdenes en la menor cantidad de caras de racks. Esta métrica (*picks per face*) es fundamental, ya que cada vez que un robot presenta una cara de un rack en la estación de picking, se quiere que el operario pueda procesar múltiples órdenes, maximizando la productividad y minimizando el movimiento de robots.
2. **Minimizar el Costo de Movimentación de Racks:** No todos los movimientos de rack tienen el mismo costo. Se debe modelar la diferencia entre traer un rack 'frío' y reutilizar un rack 'caliente':
 - **Costo de Rack Frío (desde Storage):** Traer un rack que no ha sido usado recientemente desde el área de almacenamiento tiene un costo de movimentación normalizado de **5 unidades**.

- **Costo de Rack Tibio (desde Storage reciente):** Traer un rack que ha sido usado recientemente desde el área de almacenamiento rápido (asumir 20 % de las posiciones de almacenamiento pero no modelar capacidad en detalle) tiene un costo de movimentación de **3 unidades**. 'Decidir una manera simple de modelar este concepto y justificarlo en el reporte'.
 - **Costo de Rack Caliente (cambio de cara):** Si un rack ya está siendo utilizado en el mismo ciclo en alguna de sus caras, rotarlo para presentar **cualquier otra de sus caras** tiene un costo de movimentación de **1 unidad** (5 veces menor). Se asume que un Rack puede ser Caliente si aparece más de una vez con distintas caras en un mismo pool de tareas devuelto por el SS.
3. **Cumplimiento de SLA:** Las órdenes con `due_date` más próximas deben tener mayor prioridad para evitar penalizaciones. El no cumplimiento del SLA de una orden debe tener un costo muy alto en la función objetivo.

Asumir que el Task Manager será responsable de ejecutar con prioridad las ordenes con SLA próximo incluidas en el "pool de tareas". Se considera que el tiempo máximo entre que el Task Manager toma una orden y la misma es pickeada es de **30 minutos**.

3. Consigna del Trabajo Práctico

Se espera que los alumnos desarrollen el trabajo en base a la información suministrada en este documento como complemento de toda la información suministrada en la clase del día **8/10/2026** donde se presentó el problema. El desarrollo deberá estar organizado en la en las siguientes cuatro fases:

3.1. Fase 1: Análisis del Problema y Requerimientos

Realizar un análisis formal del problema.

- Abstractar y definir claramente las entidades del problema.
- Formalizar la función objetivo.
- Identificar las restricciones principales del sistema (e.g., cantidad de órdenes a entregar por ciclo, disponibilidad de stock).

3.2. Fase 2: Exploración de Datos y Enfoques de Solución

Antes de proponer un algoritmo, explorar los datos provistos (`stock.json` y el backlog de órdenes `backlog.json`).

- Analicen la distribución del stock. POr ejemplo, ¿Hay ítems muy redundantes? ¿Están concentrados o dispersos? ¿Cuáles son los horarios de mayor caída y vencimiento de órdenes?
- Basado en este análisis, discutan la factibilidad de diferentes enfoques de solución. ¿Sería viable un enfoque de programación entera? ¿Algún tipo de heurística constructiva o metaheurísticas? Justifiquen su razonamientos.

- Incluir en el informe gráficos y visualizaciones que permitan corroborar las conclusiones analizadas.

3.3. Fase 3: Propuesta de Arquitectura

Proponer un diseño tentativo de Arquitectura para el módulo WES y la disponibilización de la información al algoritmo SS y su interacción con los otros componentes como el Task Manager o el Backlog del WMS.

3.4. Fase 4: Propuesta de un Algoritmo para el SS

Diseñen y describan en detalle un algoritmo eficiente para el módulo Shelf Selection.

- Presenten el pseudo-código del algoritmo.
- Justifiquen las decisiones de diseño. ¿Por qué eligieron esa estrategia?
- Analicen la complejidad computacional de su propuesta. ¿Es factible ejecutarlo en el ciclo de 5 minutos del Task Manager?

3.5. Fase 5: Métricas de Evaluación de Performance

Proponer un conjunto de métricas o **KPIs (Key Performance Indicators)** para evaluar la performance del algoritmo propuesto. Estas métricas deben ir más allá de la función objetivo y medir el impacto global del SS en el sistema de Outbound.

- **Métricas de Eficiencia del SS**
- **Métricas de Productividad global**
- **Métricas de Nivel de Servicio y cumplimiento de SLAs**
- **Métricas de Calidad Técnica del desarrollo**

Ejecutar la simulación del sistema para medir estos KPIs a lo largo del tiempo, es decir, durante las 24 horas de backlog. Generar de manera aleatoria (con distribución uniforme [1000, 2000]) el parámetro **N** (cantidad de órdenes que necesita procesar el TM). Generar también de manera aleatoria en la simulación las órdenes **pendientes** (distribución uniforme de hasta 100 tareas)

4. Entregables

1. El desarrollo de las cinco fases descritas en la consigna, presentados en un informe estructurado.
2. Justificar muy bien las decisiones tomadas, simplificaciones del problema y estrategias de resolución.
3. El código fuente del algoritmo propuesto, con comentarios alto nivel que permitan seguir la lógica propuesta.
4. Fecha tope de entrega: **27/10/2025 23:59hs** por el campus de la materia